

Vulnerability Report: Code Execution via Buffer Overflow

Overview:

A password of excessive length can cause a buffer overflow in PHP, which can result in a remote code execution vulnerability in PHP versions 7.4.x through 7.4.30, 8.0.x through 8.0.20, and 8.1.x through 8.1.7, when the pdo_mysql extension is used with the mysqlnd driver and the host and password for the connection, are supplied by a third party.

Vulnerability Details:

There are various situations in which an attacker can set any database server in real online applications, such as with CMS install scripts or remote database administration scripts. This report examines the possibility of remote database administration using a straightforward script to exploit this vulnerability.

A brief explanation of each step:

1. Off-by-one Heap Overflow: This is a type of buffer overflow vulnerability that occurs when a program writes data beyond the bounds of a buffer allocated on the heap. In this case, the vulnerability is due to a programming error where a single byte beyond the buffer is being overwritten.
2. POST Parsing: POST parsing is the process of parsing data sent to a web server as part of an HTTP POST request. We can use this step to prepare the heap in a specific way that will facilitate the exploitation of the off-by-one heap overflow vulnerability.
3. Fastbin Attack: Fastbins are a type of memory allocator used by the zend engine to manage heap memory. The fastbin attack takes advantage of the way fastbins are implemented in order to overwrite memory with arbitrary data.
4. Memory Leak Using zend_string Overlap: A memory leak occurs when a program loses track of a block of memory it has allocated, leading to a gradual depletion of memory resources. We are exploiting a vulnerability in the zend engine's string management to cause a memory leak.
5. Code Execution with sapi_module Overwrite: sapi_module (Server Application Programming Interface module) is a structure used by the PHP runtime to interact with the web server. By overwriting the sapi_module structure, we can execute arbitrary code with the privileges of the web server process.

Impact of the Vulnerability

In the context of PHP, a buffer overflow can have serious consequences if an attacker is able to exploit it to execute malicious code. This could allow the attacker to compromise the security of the web server hosting the PHP application, potentially leading to sensitive information such as passwords and credit card numbers being stolen. Additionally, if the web server is hosting multiple websites, an attacker could use the buffer overflow to gain access to all of them.

Proof-of-Concept Approach:

Using a hosted VM is preferred, same pc can also be used

1. Download vulnerable versions of django-navbar-client PHP from its github repository in commit history. [Github](#)
2. Download the request library.
3. Create a text file named license.txt. Configured as an Apache2 module, PHP7 proof of concept. the following configure command was used to build:

```
./configure --with-bz2 --with-zlib --with-apxs2=/usr/local/apache2/bin/apxs  
--with-mysqli=mysqlnd --enable-pdo --with-pdo-mysql=mysqlnd --enable-sockets  
--enable-mbstring --with-curl --enable-so
```

4. While installing using

```
make && sudo make install
```


Install any dependencies which are prompted during the installation process
5. Install if already not installed apache2 and run it to host the phpMyAdmin using python script for the rogue Mysql server.
6. Execute the exploit Proof-of-Concept python program to print the extracted heap address.

Setting a breakpoint on the php_output_op function and monitor its execution. RDI has a string with a bash command, and RCX points to the system. And the exploit is successful with details of Proof-of-Concept python program as follows:

1. Investigation of the bugs

The mnd_pemalloc function is used to allocate memory for the pfc->cmd buffer.buffer during the establishment of the mysql connection. Memory is allotted for permanent connections via mnd_pemalloc. MySQL database file ext/mysqlnd/mysqlnd_protocol_frame_codec.c

Persistent variable utilises zend_malloc and allocates memory to the glibc heap if it is set to 1. Persistent flags for mysql connections can be set to 0 or 1 by various classes and

functions during setup. For instance, the PDO class consistently sets persistent to 1. Pfc->cmd buffer.length is 0x1000 by default.

To perform authentication via the "mysql_clear_password" method, the server sends an auth switch packet during the authorization procedure. Client agrees to it and sends server a clear-text password. The bug appears here. Mysqlnd_wireprotocol.c in the ext/mysqlnd directory:

We desire buffer to be located on the PHP heap (not glibc heap). Send a password that is longer than 0x1000 when using the mnd_malloc function to allocate memory. Set p pointer to buffer pointer shifter by MYSQLND_HEADER_SIZE (4 bytes) and copy packe->tauth data len bytes to it. Four bytes after the buffer pointer are overwritten. Because the client delivers a password string with a null termination,we take note that the fourth byte is always null.

2. memory manager for PHP

Some understanding of the PHP memory manager is helpful when developing an exploit for a heap overflow.

In contrast to the glibc allocator, the PHP 7 allocator lacks forward/backward consolidation as well as size/prev size metadata for freed blocks.

PHP employs a method akin to glibc fastbin to store memory blocks of tiny size (less than 3072). Using the zend_mm_free_slot structure, free memory blocks of the same size are kept in a singly linked list.

Pages store memory blocks that are bigger than 3072 bytes. A memory block's size is rounded up to be multiples of the page size and can take up multiple pages, with each page having a size of 4096 bytes. It is aligned to 0x1000 at the page address. The last page may contain some unused RAM when storing a buffer on many pages. A memory block of size 0x1f00, for instance, requires two pages, with 0x100 wasted bytes on the last page. In a chunk of 512 pages, pages are kept.

Utilize the PHP allocator fastbin attack technique. Because the fourth byte is always null, we can only overwrite a maximum of 4 bytes with the bug. Having an address, next free block points to the next free memory block in a linked list. We are looking for a means to create a stable exploit without using brute force and to get around ASLR.

The simplest technique to get around address randomization is to rewrite only the last byte of the smallbin page because we can't predict the address on the server where it is allocated (with null-byte). As a result, we convert a 4-byte overrun into an off-by-one overflow.

The size of the large chunk is rounded to be multiples of 0x1000, and the address is aligned to 0x1000. After the buffer, the last grabbed page may have some free memory, and an overflow will write into this free memory, leaving no room for attack. We multiply the buffer size by 0x1000 to get rid of the surplus space left behind after the buffer. In this instance, writing past the buffer writes to the following page. The only method to change the next free block pointer is to transmit a password that is multiples of the page size and place the page allocated for the smallbin linked list directly after the pages allocated for the mysql password. The following step is to set up the PHP heap so that the memory block for the password will always be allocated before the memory block for the smallbin.

3. POST parsing is used to prepare the heap.

Locate the lines of code where memory is set aside for POST variables after some troubleshooting. One POST var is parsed by PHP using three memory allocations.

After the memory block used for parsing is released, this allocation will be temporary.

Memory for the string POST-variable is allocated twice more.

For page allocation, send a lot of POST variables. Python scripting language code.

Many smallbins are empty when POST processing begins; thus, new pages for smallbin linked lists are allocated as a result of memory being taken from empty smallbins.

```
26     payload.append(('hi1', 'T' * (135 - 24 - 3)))
27     payload.append(('hi2', 'T' * (135 - 24 - 3)))
28     payload.append(('hi3', 'T' * (135 - 24 - 3)))
29     payload.append(('hi4', 'T' * (135 - 24 - 3)))
30     payload.append(('hi5', 'T' * (135 - 24 - 3)))
31     payload.append(('hi6', 'T' * (135 - 24 - 3)))
32     payload.append(('hi7', 'T' * (135 - 24 - 3)))
```

POST variables should be sent to a smallbin to take memory. This bin contains memory that is retrieved in sizes 130 to 160.

```
47
48     for i in range(N):
49         if i % 2 == 0 and i < int(N / 1.5):
50             continue
51         payload.append(('key%s' % i, 'helloworld'))
52
```

we can see that the memory set aside for POST variables that was previously positioned before this smallbin is now free. After POST parsing, the PHP script moves on to the lexical analysis and compilation of Zend VM opcodes steps. They steal a few unclaimed pages. Since there are just about 50 lines of code in our script, parsing doesn't require much RAM.

Now set aside space for the MySQL password.

The best fit algorithm is used by the PHP7 allocator while looking for huge blocks of memory. To fill the most appropriate gap, the start page is looked up.

Allocator has "gaps" of unoccupied pages of various lengths.

Password sent with length 0x8ffd and "gap" taken from page

4. both a memory leak and a fastbin attack

In PHP7, the string content is added to the end of the zend string structure. "echo" should be used to replace the value of "len" and the output variable. With the use of heap overlapping techniques, structure overwrite can be accomplished.

When POST variables are parsed, Smallbin is ready. Keep in mind that PHP uses two memory allocations for one variable while parsing POST input. To allocate memory from smallbin, send 7 POST variables from 'hi1' to 'hi7'. Therefore, $7 * 2 = 14$ portions are assigned. Following these allocations, chunk 15 becomes free and the top of the linked list for smallbin 12 (smallbin 12). Chunks 1 and 2 were released after the "null" value for the 'hi1' variable was overwritten in the POST. Chunk 1 refers to Chunk 15. (with address 0x7f6eac6c98c0). Before a heap overflow, dump the smallbin 12 linked list.

Memory allocation is caused by taking more memory from this smallbin.

Get some pieces out of smallbin by finding a technique to do so. Do not forget that we have a mysql connection open to a server that we have complete control on. In the code that interprets server responses and executes SELECT commands, the PHP mysqlnd client makes several emalloc calls. Simply read the explanation of the MySQL protocol to create a malicious MySQL server.

For memory allocation of any size, use the macro ZVAL_STRINGL. Do not forget that the zend string structure's string data is added at the end. PHP sets aside $N + 25$ bytes of RAM when creating strings with lengths of N bytes. Python exploit source code

5. making code to execute

To write arbitrary data to memory, use the fastbin attack. To assign chunks 1 through 10 in smallbin, send 5 POST variables from "hi1" to "hi5". The smallbin linked list's head is changed to chunk 11. Observe that the "hi5" variable contains a false zend_mm_free_slot structure.

```
38 fake_chunk = ('Y' * 0x48).encode('utf-8')
39 fake_chunk += struct.pack('<Q', sapi_ub_write - 24) #next_free_
40 fake_chunk += ('T' * (135 - 24 - 3 - len(fake_chunk))).encode('utf-8')
41
42 payload.append(('hi1', 'T' * (135 - 24 - 3)))
43 payload.append(('hi2', 'T' * (135 - 24 - 3)))
44 payload.append(('hi3', 'T' * (135 - 24 - 3)))
45 payload.append(('hi4', 'T' * (135 - 24 - 3)))
46 payload.append(('hi5', fake_chunk))
```

To free some RAM, replace a few POST variables with "null." With regard to the smallbin linked list, chunk 1 points to chunk 11.

```
64 payload.append(('hi1', 'null'))
65 payload.append(('hi2', 'null'))
66 payload.append(('hi3', 'null'))
67 payload.append(('hi4', 'null'))
```

The final byte of chunk 1's next free slot pointer, from 40 to 00, is overwritten with an off-by-one overflow.

The bogus zend_mm free slot structure in chunk 10 is now the next free slot pointer in chunk 1.

Context.out.data is the string that is supplied for output, and sapi_module.write is the callback that is contained in the global array. If output buffering is not deactivated (in the case of Apache2), output will be processed by the output handler. It makes an ideal target for "system" address overwriting. Get a synthetic chunk structure with overwrite addresses ready.

After an off-by-one overwrite, dump smallbin.

Allocations should be made in the MySQL connection to return the destination address.

Solution: Update the PHP version from 7.4.x to 7.4.30, 8.0.x to 8.0.20, and 8.1.x to 8.1.7,